

Réseaux pour le cloud

Jour 3 — Transporter et protéger : TCP, HTTP, TLS, load balancing, pare-feu, VPN

La journée la plus dense de la semaine — tout ce qui touche vos futurs ALB et security groups

Objectifs de la journée

À la fin de cette journée, vous serez capable de :

- Expliquer la différence **TCP / UDP** : ports, handshake, fiabilité.
- Lire un échange **HTTP/1.1** (méthodes, codes, en-têtes) et situer les apports de **HTTP/2** ; expliquer cookies et sessions dans les grandes lignes.
- Décrire **TLS** : certificats, autorités (CA), chaîne de confiance, handshake simplifié, Let's Encrypt.
- Distinguer un load balancer **L4** d'un **L7**, expliquer health checks et algorithmes de répartition.
- Distinguer pare-feu **stateful** et **stateless**, lire une règle iptables/nftables, configurer **ufw**.
- Expliquer le principe d'un **VPN**, site-à-site vs poste-à-site, WireGuard.

Plan de la journée

0. **Échauffement subnetting** (10 min — 3 calculs, rituel du matin).
1. **TCP vs UDP** — ports, handshake, fiabilité.
2. **HTTP/1.1 et HTTP/2** — méthodes, codes, en-têtes, cookies.
3. **TLS** — certificats, CA, chaîne de confiance, Let's Encrypt.
4. **Répartition de charge** — L4 vs L7, health checks, algorithmes.
5. **Pare-feu** — stateful vs stateless, iptables/nftables, ufw.
6. **VPN** — site-à-site, poste-à-site, WireGuard.
7. Récap + quiz.

Démo fil rouge : un échange TLS mis à nu avec `curl -v` et `openssl s_client`.



Échauffement — 3 calculs chrono (10 min)

1. `10.128.71.9/18` : adresse réseau et broadcast ?
2. Découpez `172.30.0.0/16` en 4 sous-réseaux égaux : listez-les.
3. Quel préfixe pour une liaison point à point de 2 hôtes ?

Correction éclair par deux volontaires au tableau — méthode posée exigée.

1. TCP vs UDP

Deux philosophies du transport

Les ports : la porte d'entrée des applications

L'adresse IP identifie **la machine** ; le **port** (0-65535) identifie

l'application sur cette machine.

Une connexion = un quadruplet : `IP source : port source → IP dest : port dest`.

Port	Service	Port	Service
22	SSH	443	HTTPS
53	DNS	3306	MySQL
80	HTTP	5432	PostgreSQL (StockLine !)
123	NTP	8000	dev FastAPI/uvicorn

- Ports < **1024** : « bien connus », réservés à root pour l'écoute.
- Le client utilise un port **éphémère** (grosso modo 32768-60999 sous Linux) — retenez-le : c'est la clé du piège NACL de vendredi.

```
ss -tlnp      # qui écoute en TCP sur ma machine ? (vu en CL-LINUX)
```

TCP : fiable, connecté, ordonné

TCP (Transmission Control Protocol) garantit que les données arrivent **complètes, dans l'ordre, sans doublon** :

- **connexion** établie avant tout échange (handshake) ;
- chaque octet est **numéroté** (numéros de séquence) ;
- le récepteur **acquitte** (ACK) ; ce qui n'est pas acquitté est **retransmis** ;
- contrôle de flux et de congestion : TCP ralentit si le réseau sature.

Le prix : de la latence (aller-retours) et de l'état à maintenir de chaque côté.

Utilisé par : HTTP(S), SSH, bases de données (PostgreSQL), SMTP...

Tout ce qui ne tolère pas la perte.

Le handshake TCP en 3 temps

CLIENT	SERVEUR
---- SYN (seq=x) ----->	« je veux parler »
<--- SYN-ACK (seq=y, ack=x+1) ----	« ok, moi aussi »
---- ACK (ack=y+1) ----->	« confirmé »
===== connexion ÉTABLIE =====	
... échange de segments + ACK ...	
---- FIN / ACK (x2) ----->	fermeture propre

À retenir : **SYN**, **SYN-ACK**, **ACK**. Vous le verrez en clair dans la démo `curl -v` et dans tcpdump.

UDP : rapide, sans promesse

UDP (User Datagram Protocol) envoie des datagrammes **sans connexion**, **sans accusé**, **sans ordre garanti** :

- pas de handshake → **zéro latence** d'établissement ;
- un datagramme perdu est **perdu** (à l'application de gérer, ou pas) ;
- parfait quand la fraîcheur prime sur la complétude.

Utilisé par : **DNS** (requêtes courtes), NTP, streaming/visio, jeux en ligne, **et QUIC/HTTP-3** (qui réimplémente la fiabilité au-dessus d'UDP).

	TCP	UDP
Connexion	oui (SYN/SYN-ACK/ACK)	non
Fiabilité / ordre	garantis	non garantis
Vitesse d'établissement	plus lent	immédiat
Exemples	HTTP, SSH, SQL	DNS, NTP, visio

2. HTTP

**Le protocole que vous administrerez toute
votre carrière**

Anatomie d'un échange HTTP/1.1

HTTP est un protocole **texte**, requête/réponse, au-dessus de TCP (port 80, ou 443 via TLS) :

REQUÊTE	RÉPONSE
GET /produits HTTP/1.1	HTTP/1.1 200 OK
Host: stockline.example.com	Content-Type: application/json
User-Agent: curl/8.5.0	Content-Length: 182
Accept: application/json	Server: nginx
	[{"id":1,"nom":"Clavier",...}]

- 1re ligne de requête : **méthode + chemin + version**.
- 1re ligne de réponse : **version + code + libellé**.
- Puis les **en-têtes** (clé: valeur), une ligne vide, et le **corps**.
- **Host:** est obligatoire : un même serveur héberge plusieurs sites (c'est aussi ce qu'utilisera l'ALB pour router en L7).

Méthodes et codes de statut

Méthodes (vous les avez pratiquées avec FastAPI en CL-PYTHON) :

- **GET** lire — **POST** créer — **PUT/PATCH** modifier — **DELETE** supprimer ;
- GET ne doit **jamais** modifier l'état (les robots suivent les GET !).

Codes de statut — retenir les familles + les vedettes :

Famille	Sens	À connaître
2xx	succès	200 OK, 201 Created, 204 No Content
3xx	redirection	301 permanente, 302 temporaire, 304 cache
4xx	erreur client	400 , 401 non authentifié, 403 interdit, 404 , 429
5xx	erreur serveur	500 , 502 Bad Gateway, 503 , 504 timeout

Réflexe d'admin : **502/504 derrière un load balancer** = le backend est en cause (mort, lent, ou health check cassé) — vous le vivrez au TP2.

Cookies et sessions en bref

HTTP est **sans état** : chaque requête est indépendante. Pour reconnaître un utilisateur entre deux requêtes :

```
réponse : Set-Cookie: session=abc123; HttpOnly; Secure  
requêtes suivantes : Cookie: session=abc123
```

- le serveur associe `abc123` à des données de **session** (panier, login) ;
- attributs de sécurité : `Secure` (HTTPS seulement), `HttpOnly` (inaccessible au JavaScript), `SameSite` (anti-CSRF) – détails en CL-SECU.

Conséquence d'architecture (importante pour le TP2) : si la session est stockée **en mémoire d'un serveur**, un load balancer qui envoie l'utilisateur sur un **autre** serveur le déconnecte → soit **sticky sessions**, soit session **externalisée** (base, cache). StockLine est sans état côté API : c'est un choix.

HTTP/2 : même sémantique, transport plus malin

HTTP/1.1 : une requête à la fois par connexion TCP → les navigateurs ouvraient 6 connexions en parallèle et ça bloquait quand même (head-of-line blocking).

HTTP/2 garde méthodes/codes/en-têtes mais change le transport :

- **binaire** (plus du texte brut sur le fil) ;
- **multiplexage** : des dizaines de requêtes simultanées sur **une seule** connexion TCP ;
- **compression des en-têtes** (HPACK) ;
- en pratique, toujours utilisé **avec TLS**.

Ce que ça change pour vous : rien dans le code, tout dans la config — c'est le **load balancer ou nginx** qui parle HTTP/2 au client (l'ALB AWS le fait par défaut). HTTP/3 = même idée sur **QUIC/UDP** ; retenez le nom, hors programme.

3. TLS

Chiffrer et prouver son identité

Ce que TLS garantit (et HTTPS = HTTP + TLS)

TLS (Transport Layer Security, successeur de SSL — le nom « SSL » survit dans le langage courant) apporte trois garanties :

1. **Confidentialité** : le contenu est chiffré, illisible en transit.
2. **Intégrité** : toute altération est détectée.
3. **Authenticité** : vous parlez bien à `stockline.example.com`, pas à un imposteur — c'est le rôle du **certificat**.

Sans le point 3, les points 1 et 2 ne servent à rien : chiffrer sa conversation avec un pirate reste une conversation avec un pirate.

Versions : **TLS 1.2** (encore courant) et **TLS 1.3** (standard actuel, handshake plus rapide). SSLv3, TLS 1.0/1.1 : **à bannir** — les scanners de CL-SECU vous le rappelleront.

Le certificat : une carte d'identité signée

Un **certificat X.509** lie une **clé publique** à une **identité**, le tout **signé par une autorité de certification (CA)** :

- **Subject / SAN** (Subject Alternative Names) : les noms couverts
(`stockline.example.com`, `*.example.com` pour un wildcard) ;
- **Validité** : dates de début et de fin (90 jours chez Let's Encrypt) ;
- **Émetteur (Issuer)** : la CA qui a signé ;
- la **clé privée** correspondante reste **sur le serveur, secrète** —
la fuite de la clé privée = certificat compromis.

Principe de la signature : la CA appose une signature cryptographique vérifiable par tous ; **personne ne peut modifier le certificat** sans invalider la signature.

Erreurs navigateur classiques : certificat **expiré**, **nom non couvert** (SAN $\neq$ domaine visité), **CA inconnue** (auto-signé).

La chaîne de confiance

```
[ CA RACINE ]           <- pré-installée dans l'OS/navigateur
  | signe                (magasin de confiance)
  v
[ CA INTERMÉDIAIRE ]    <- ex. Let's Encrypt "R11"
  | signe
  v
[ CERTIFICAT SERVEUR ]  <- stockline.example.com
```

Le navigateur vérifie **la chaîne complète** : serveur → intermédiaire
→ une racine qu'il connaît déjà. Il suffit d'**une** racine de confiance pour valider des millions de sites.

Piège d'admin n°1 : oublier de servir le certificat **intermédiaire**
→ ça marche dans certains navigateurs (cache) et pas dans d'autres.

Le handshake TLS, version simplifiée

```

CLIENT                                SERVEUR
| -- ClientHello (versions, algos, SNI) --> |
| <-- ServerHello + CERTIFICAT ----- |
|   [vérif chaîne + nom + dates]         |
| <==== échange de clés (éphémère) =====> |
| ===== canal chiffré établi ===== |
| ... et SEULEMENT ensuite : HTTP ...   |

```

Points à retenir :

- le **SNI** (Server Name Indication) : le client annonce le nom demandé **dans le ClientHello** → un serveur peut présenter le bon certificat parmi plusieurs (même principe que **Host:**, mais au niveau TLS) ;
- les clés de session sont **éphémères** : compromis aujourd'hui ne déchiffre pas hier (forward secrecy) ;
- tout HTTP passe **après** : un load balancer L4 ne voit donc **rien** du contenu, un L7 doit **terminer** le TLS pour lire les requêtes.

Let's Encrypt : des certificats gratuits et automatisés

Let's Encrypt est une CA gratuite qui a banalisé HTTPS :

- certificats de **90 jours**, renouvelés **automatiquement** ;
- protocole **ACME** : la preuve de contrôle du domaine est automatisée — défi **HTTP-01** (déposer un fichier sur le serveur web) ou **DNS-01** (créer un enregistrement TXT — tiens, le TXT d'hier) ;
- client type : **certbot** (vous l'utiliserez avec nginx).

```
sudo certbot --nginx -d stockline.example.com # obtention + config nginx
sudo certbot renew --dry-run                 # tester le renouvellement
```

Anticipation cloud : chez AWS, **ACM** joue ce rôle (certificats gratuits, renouvelés seuls, validés par DNS via Route 53) pour vos ALB et CloudFront — CL-AWS1 J7. La logique ACME/validation est la même.



Démo 3-1 — Un échange TLS mis à nu

Fichier : `demos/04-cl-reseau/demo-3-1-tls-en-clair.md`

En direct sur la VM :

- `curl -v http://...` : requête et réponse HTTP **en clair**, ligne par ligne ;
- `curl -v https://...` : le handshake TLS apparaît **avant** le HTTP (version TLS, certificat, ALPN/HTTP-2) ;
- `openssl s_client -connect ...:443` : la **chaîne de certificats** brute ;
- `openssl x509 -text` : lire un certificat (SAN, dates, émetteur) ;
- provoquer des erreurs : mauvais nom, certificat expiré (sites de test dédiés).

Ensuite : exercice 3-1, vous analysez un échange complet, seul.



Exercice 3-1 – Analyser un échange HTTP/TLS (40 min)

Fichier : `exercices/04-cl-reseau/exercice-3-1-analyse-http-tls.md`

À partir d'une sortie `curl -v` complète (fournie et annotable) :

- repérer les phases : TCP, TLS, HTTP ;
- identifier version TLS, émetteur du certificat, SAN, dates ;
- lire la requête (méthode, chemin, en-têtes) et la réponse (code, redirection, cookie) ;
- expliquer une erreur de certificat fournie en bonus.

Puis quelques manipulations réelles sur la VM pour vérifier vos réponses.

4. La répartition de charge

Un service, plusieurs serveurs

Pourquoi un load balancer ?

```
clients --> [ LB ] --> [ serveur app 1 ] OK
                  +--> [ serveur app 2 ] OK
                  +--> [ serveur app 3 ] KO (retiré)
```

Trois raisons d'exister :

1. **Répartir** la charge entre plusieurs serveurs identiques.
2. **Tolérer les pannes** : un serveur mort est retiré du pool (grâce aux health checks).
3. **Point d'entrée unique** : une seule IP/nom public, du TLS centralisé, des serveurs cachés derrière.

C'est l'ALB de votre TP2 : StockLine tournera sur plusieurs instances EC2 derrière un load balancer.

L4 vs L7 : à quel étage travaille le répartiteur ?

	LB L4 (transport)	LB L7 (application)
Il voit	IP + ports TCP/UDP	les requêtes HTTP complètes
Décisions possibles	répartir des connexions	router par chemin , Host , en-tête
TLS	passe tel quel (passthrough)	terminé au LB (il déchiffre)
Performance	très rapide, protocole-agnostique	plus riche, plus coûteux
Chez AWS	NLB	ALB ← votre TP2

Exemples de règles L7 : `/api/*` → groupe FastAPI, `/static/*` → serveurs de fichiers ; `Host: admin.example.com` → backend d'admin.

Conséquence TLS (vue ce matin) : pour router sur l'URL, le LB **doit** terminer le TLS. Le certificat vit alors **sur le LB** (ACM sur l'ALB), plus sur les serveurs.

Health checks et algorithmes

Health check : le LB sonde chaque backend à intervalle régulier
(ex. `GET /sante` toutes les 10 s — l'endpoint existe déjà dans StockLine !) :

- N échecs consécutifs → serveur **retiré** du pool (unhealthy) ;
- N succès → **réintégré**. Personne ne redémarre rien à la main :
automatiser, superviser, sécuriser.

Algorithmes de répartition courants :

- **round robin** : chacun son tour (le défaut, sain pour des serveurs identiques) ;
- **least connections** : au moins chargé (sessions longues) ;
- **hash IP source** : le même client revient au même serveur
(une forme de stickiness).

Bien choisir son health check : sonder `/sante` (qui teste l'accès base)
plutôt que `/` (qui peut répondre 200 alors que la base est morte).

5. Le pare-feu

Qui entre, qui sort

Stateless vs stateful : LA distinction à maîtriser

Un pare-feu filtre les paquets selon des règles (IP, port, protocole, sens).

Deux philosophies :

- **Stateless** : chaque paquet est jugé **isolément**, sans mémoire.
Il faut écrire les règles **dans les deux sens** (l'aller ET le retour).
- **Stateful** : le pare-feu **suit les connexions**. Si la connexion sortante est autorisée, **les paquets de réponse passent automatiquement**.

```
STATEFUL : OUT vers 203.0.113.5:443 autorisé  
=> réponses de 203.0.113.5:443 acceptées (connexion connue)
```

```
STATELESS : OUT vers :443 autorisé  
+ il FAUT une règle IN pour les réponses  
(src :443 vers mes ports éphémères 32768-65535)
```

Retenez dès maintenant : **security group = stateful, NACL = stateless**.

Vendredi, cette slide vaudra de l'or.

iptables / nftables : savoir lire

Sous Linux, le filtrage est fait par **netfilter**, piloté historiquement par **iptables**, aujourd'hui par **nftables**. Objectif ici : **lire**, pas écrire.

```
# iptables : autoriser SSH entrant, et les connexions déjà établies
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -P INPUT DROP          # politique par défaut : tout jeter
```

```
# nftables : la même logique, syntaxe moderne
nft list ruleset                # lire les règles actives
# ... tcp dport 22 accept
# ... ct state established,related accept
```

- La ligne **ESTABLISHED,RELATED** = **c'est ça, le stateful** (conntrack).
- **-P INPUT DROP** : le **défaut ferme**, on ouvre explicitement — le bon réflexe partout, y compris dans le cloud.

ufw : le pare-feu au quotidien sur Ubuntu

ufw (Uncomplicated Firewall) est une façade simple au-dessus de netfilter — c'est lui que vous utiliserez sur la VM et au TP1 :

```
sudo ufw default deny incoming      # défaut : rien n'entre
sudo ufw default allow outgoing     # défaut : tout sort
sudo ufw allow 22/tcp               # SSH (AVANT enable, sinon adieu la VM !)
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow from 203.0.113.50 to any port 5432 proto tcp # source limitée
sudo ufw enable
sudo ufw status verbose             # relire ce qui est actif
```

- ufw est **stateful** : pas de règle « retour » à écrire.
- Règle d'or n°1 : **autoriser SSH avant d'activer**.
- Règle d'or n°2 : le port PostgreSQL **jamais ouvert à tous** — toujours restreint à une source (au TP2 : au security group de l'app).



Exercice 3-2 — Écrire des règles de pare-feu (40 min)

Fichier : `exercices/04-cl-reseau/exercice-3-2-regles-pare-feu.md`

Mise en situation : la VM StockLine du TP1 (nginx en 80/443, API en 8000 locale, PostgreSQL en 5432, SSH d'admin depuis une seule IP) :

- écrire la politique ufw complète (défauts + règles, dans le bon ordre) ;
- justifier chaque règle en une phrase (« qui, vers quoi, pourquoi ») ;
- version stateless : écrire les règles **aller et retour** du seul flux HTTPS, pour toucher du doigt ce que le stateful vous épargne ;
- lecture : 4 règles iptables fournies à traduire en français.

C'est le brouillon direct de vos futurs security groups.

6. VPN

Un tunnel privé sur un réseau public

Le principe : encapsuler et chiffrer

Un **VPN** (Virtual Private Network) crée un **tunnel chiffré** entre deux points à travers un réseau non fiable (Internet) :

```
[ paquet privé 10.0.1.5 -> 10.8.0.2 ]      <- invisible  
encapsulé dans :  
[ paquet public 203.0.113.9 -> 198.51.100.4, CHIFFRÉ ]  <- seul visible
```

- Les adresses **privées** voyagent **dans** un paquet public chiffré.
- De l'extérieur, on voit deux extrémités qui échangent du bruit.
- Côté systèmes : le tunnel apparaît comme **une interface réseau de plus** (`wg0`, `tun0`) avec **sa route** dans `ip route` — tout ce que vous savez déjà s'applique.

Site-à-site vs poste-à-site, et WireGuard

Deux usages à ne pas confondre :

	Site-à-site	Poste-à-site (remote access)
Relie	deux réseaux entiers	un poste à un réseau
Extrémités	deux passerelles (routeurs)	client sur le laptop ↔ passerelle
Usage type	agence ↔ siège, datacenter ↔ VPC	télétravail, admin
Chez AWS	Site-to-Site VPN (vendredi)	Client VPN

WireGuard (démonstration conceptuelle) : le VPN moderne –

~4000 lignes de code, dans le noyau Linux, cryptographie actuelle, configuration en quelques lignes (paires de clés + `AllowedIPs`, qui n'est... qu'une liste de **CIDR** : encore le calcul de lundi).

Alternatives historiques : IPsec (le standard des site-à-site managés), OpenVPN.

Récap des points clés du jour

- **TCP** : connecté, fiable, ordonné — **SYN, SYN-ACK, ACK** ;
UDP : sans connexion, rapide, sans garantie (DNS, visio, QUIC).
- Ports : IP = machine, **port = application** ; clients sur ports éphémères.
- **HTTP** : méthodes + codes (familles 2/3/4/5xx, 502-504 = regarder le backend) ;
cookies = état par-dessus un protocole sans état ; HTTP/2 = multiplexage.
- **TLS** : certificat signé par une **CA** → **chaîne de confiance** ;
SNI, forward secrecy ; **Let's Encrypt/ACME** automatise tout (ACM chez AWS).
- **LB** : L4 = ports, **L7 = HTTP** (terminaison TLS) ; health checks sur
`/sante` ; round robin / least connections.
- **Pare-feu** : **stateful** (suit les connexions) vs **stateless**
(règles aller **et** retour) ; ufw : défaut fermé, SSH d'abord.
- **VPN** : tunnel chiffré ; site-à-site (réseaux) vs poste-à-site (personnes).



Quiz — questions 1 et 2

Question 1

Citez, dans l'ordre, les trois messages du handshake TCP.

Question 2

Le DNS utilise majoritairement UDP pour ses requêtes. Donnez deux raisons qui rendent UDP adapté à cet usage.



Quiz — questions 3 et 4

Question 3

Quelle est la différence entre un code HTTP **301** et un **302** ?

Et que doit vous évoquer un **502** renvoyé par un load balancer ?

Question 4

Quel en-tête de réponse HTTP dépose un cookie chez le client, et quel attribut empêche ce cookie de circuler en HTTP non chiffré ?



Quiz — questions 5 et 6

Question 5

Dans la chaîne de confiance TLS, qu'est-ce qui permet à votre navigateur de faire confiance à un site qu'il n'a jamais visité ?

Question 6

Pourquoi un load balancer doit-il « terminer » le TLS pour router selon l'URL demandée ?



Quiz — questions 7 et 8

Question 7

À quoi sert un health check, et pourquoi vaut-il mieux sonder `/sante` (qui teste la base) que `/` sur StockLine ?

Question 8

Un pare-feu **stateless** autorise le flux sortant vers `203.0.113.5:443`.

Quelle règle supplémentaire faut-il écrire pour que la réponse revienne, et pourquoi un pare-feu **stateful** s'en passe-t-il ?



Quiz — questions 9 et 10

Question 9

Donnez les commandes ufw pour : tout refuser en entrée par défaut, autoriser SSH, autoriser HTTPS — dans un ordre qui ne vous coupe pas l'accès à la machine.

Question 10

Quelle est la différence entre un VPN **site-à-site** et un VPN **poste-à-site** ? Lequel utiliserez-vous vendredi pour relier un datacenter à un VPC ?

Jour 4 — Le réseau dans le cloud : VPC, security groups, interconnexion

Demain, tout converge : le CIDR de lundi devient un VPC, les routes
de mardi

deviennent des tables de routage de sous-réseaux, les pare-feu
d'aujourd'hui

deviennent security groups et NACL. Et vous dessinerez **le plan
d'adressage**

complet de StockLine — celui que vous construirez pour de vrai au